SessionBound: Database-Enforced Task-Bound Sessions for Enterprise AI Agents

Minmin Wu
China Telecom Global Limited
wuminmin@chinatelecomglobal.com

***Abstract*—Enterprise AI agents can write useful exploratory SQL for audit, compliance, and operational analysis, but raw database access is over-privileged and fixed SaaS screens are too rigid for temporary business tasks. This paper presents SessionBound, a database-enforced task-bound session model for enterprise AI agents. A control plane turns an approved task into a signed token carrying task scope, safe views, denied fields, TTL, operation policy, budgets, and policy-version claims. The PostgreSQL runtime, SessionBoundDB, binds the token to a short-lived credential and enforces the approved boundary through safe-view resolution, read-only SQL policy, disclosure accounting, minimum-group aggregate protection, schema-drift checks, and receipts.**

SessionBoundDB implements an accounting-complete wrapper reference path and a native PostgreSQL hook/executor hardening path. The evaluation validates 24 canonical scenarios and a 140-case adversarial SQL suite, blocking 126 boundary-violation cases while allowing and accounting for 14 safe-view analytical cases. A stronger RLS+safe-view+short-credential+audit baseline shows that conventional database controls cover important pieces but do not provide task-token binding, cumulative disclosure budgets, safe-view drift invalidation, or receipt chains. On the default seed, full SessionBound has 17.4–18.6 ms p50 latency in the wrapper path. Diagnostic scale benchmarks show that structural guarding is sub-millisecond, while unoptimized result accounting becomes the bottleneck for large detail-release stress settings. These results support SessionBound as a task-session contract for database-enforced AI-agent governance, while identifying executor-level accounting optimization as future work.

***Index Terms*—AI agents, access control, database security, dependable computing, secure computing, authorization, SQL security, data governance, auditability.**

I. Introduction

AI agents change how enterprise users interact with data. A user no longer needs to know which report exists, which dashboard to open, or which API endpoint to call. Instead, the user can ask an agent:

> Analyze June 2026 travel reimbursements for the Sales department. Find unusual merchants, repeated claims, employee-level monthly totals, and claims near approval thresholds. Exclude salary and bank data.

This is exactly the kind of task agents are good at: temporary, exploratory, cross-table, and under-specified. It is also exactly the kind of task traditional enterprise software handles poorly. Building a permanent SaaS page or API for every low-frequency internal analysis task is expensive. Giving the agent raw database access is unsafe. Relying on prompts is not a security boundary. Relying on the application layer alone becomes fragile when SQL is generated dynamically.

Existing systems cover important pieces: delegated identity and OAuth scopes [1], [2], task-scoped tool authorization [3], data-product governance [4], [5], database-native policy enforcement [6], [7], [8], and large-scale relationship authorization [9]. They do not, by themselves, turn one approved analytical task into a temporary SQL session with safe views, denied fields, budgets, and receipts. That is the gap SessionBound addresses.

This paper argues for the following principle: business users approve tasks, not database policies. Agents generate SQL, but databases enforce the approved boundary. This paper frames the problem as a database-enforced authorization boundary for enterprise AI agents, rather than as an agent-planning or multi-agent coordination problem. SessionBound treats an approved enterprise task as the unit of database execution, rather than treating identity, role, relation, or individual query as the sole authorization unit. The agent can try. The database decides.

SessionBound separates a task control plane that approves and signs structured claims, an agent execution layer that proposes SQL, and a database runtime that deterministically enforces safe views, scope, denied fields, operation limits, budgets, and receipts.

This makes SessionBound a contract layer between enterprise approval and agentic database execution. The control plane records what the organization approved; the runtime enforces how that approval may be spent through SQL. This contract is not a database policy written by a DBA, nor a natural-language instruction interpreted by an LLM. It is a structured, signed, and accountable representation of an approved business task that both the agent runtime and the database can interpret deterministically.

SessionBoundDB is our PostgreSQL-based reference runtime for SessionBound. It does not ask an LLM whether a query is safe. It does not depend on the agent correctly interpreting the task. It does not trust mutable session variables set by the agent. Instead, it validates a signed task token, exposes only registered safe views, rejects

raw table access and denied fields, tracks query and disclosure budgets, and records receipts for evaluated allowed executions and bound-runtime denial decisions.

The novelty is not any single primitive. It is the binding of task approval, credential identity, safe-view versioning, SQL surface, cumulative disclosure budget, and receipt semantics into one database-execution session object: enterprise task approval is compiled into an execution-time database session contract rather than scattered across separate credentials, views, policies, and audit logs.

A. Contributions

This paper makes five contributions.

1. Task-bound database sessions. We identify the approved enterprise task, rather than only identity, role, relation, or query, as the stateful authorization object for agentic SQL.
2. Signed control-plane contract. Templates, applications, approvals, grants, TTLs, budgets, and task tokens become database-consumable structured claims rather than natural language.
3. Composed session object. Credential-token matching, safe-view versioning, exposed-column hashing, disclosure budgets, minimum-group policy, and receipts are bound into one session object.
4. PostgreSQL prototype with wrapper and native hardening paths. We implement an accounting-complete wrapper/API reference path and a native PostgreSQL hook/executor path for direct safe-view SQL, prepared statements, cursor/`FETCH`, `COPY (SELECT)`, utility checks, and executor-level accounting.
5. Evaluation against stronger baselines and attacks. We evaluate canonical behavior, adversarial SQL, credential-token binding, drift invalidation, a strong RLS+safe-view+short-credential audit baseline, ablations, scale behavior, and hook-only structural cost.

II. Motivation: Enterprise Tasks Are Approved Above the Database

SessionBound targets temporary enterprise analysis that is too specific for a permanent application screen and too sensitive for raw database access: reimbursement audits, vendor-concentration checks, compliance-log reviews, or operational investigations. These tasks need joins, aggregation, ranking, drill-down, and follow-up SQL, but the exact query plan is not known when the task is approved.

For example, a business user may request a June 2026 Sales travel reimbursement audit that excludes salary, phone, and bank-account fields, allows 20 SQL attempts and 5,000 result rows, and expires after 30 minutes. A manager or data owner can approve this task without writing RLS predicates or database grants. A data platform can expose safe views. A compliance policy can tighten budgets. The agent can then generate SQL inside the approved boundary, while the database rejects raw tables, denied fields, unsafe operations, and over-budget attempts.

III. Threat Model

SessionBound assumes the upper-layer agent is useful but not trusted. The agent may be confused, overconfident, prompt-injected, malicious, or simply wrong. The database boundary must remain meaningful even when the agent tries the wrong thing.

A. Trusted components

For this prototype, we trust:

- the task control plane that defines templates, evaluates grants, records approvals, and signs task tokens;
- the credential broker that issues short-lived database credentials;
- the database runtime that validates tokens and enforces policies;
- the cryptographic signing key or signing service used for task tokens;
- the integrity of registered safe views, runtime functions, and DB-local append-only audit storage.

In production, signing should use a KMS, JWKS, hardware-backed key service, or enterprise identity infrastructure. The prototype uses a simpler signing path. Receipt-chain tamper evidence assumes trusted DB-local append-only storage; DBA-resistant deployments should anchor chain heads to external WORM storage, SIEM, transparency logs, or enterprise audit infrastructure.

B. Untrusted or partially trusted components

We do not trust:

- the agent's prompt;
- the agent's reasoning;
- the agent's SQL generation;
- the agent's natural-language interpretation of the task;
- user-provided or database-provided text that may contain prompt injection;
- ordinary client-controlled session variables;
- application-layer filtering as the final boundary.

This is the main difference from designs that require an LLM to infer the precise intended operation from natural language. SessionBound does not ask the database to understand the user's natural-language intent. The database consumes structured claims from a signed token.

C. In-scope threats

SessionBoundDB is designed to mitigate:

- Sensitive field access: attempts to read salary, bank account, phone, identity number, credentials, or private notes.
- Raw table access: attempts to bypass safe views and query application schemas directly.
- Scope escape: attempts to read other tenants, months, departments, projects, or users.
- Unauthorized mutations: write, DDL, or other high-impact operations outside controlled commands.
- Prompt injection effects: data or user text instructs the agent to ignore policy or reveal secrets.

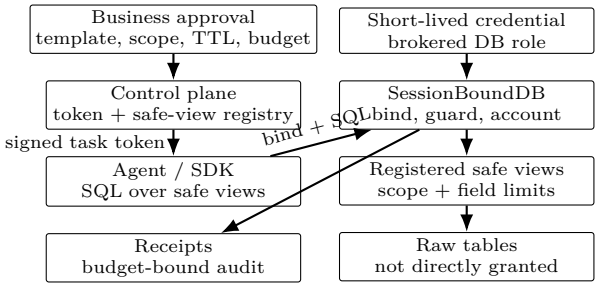

Fig. 1. SessionBound architecture. The approved enterprise task becomes a signed database-session contract; agents can attempt SQL, but the database runtime enforces the bound task state.

- Budget evasion: repeated queries, pagination, or alternate SQL forms attempt to disclose more detail rows than the task allows.
- Credential-token mismatch: attempts to combine one runtime credential with another task token.

D. Out of scope

The prototype does not solve:

- malicious DBAs, storage administrators, or compromised database hosts;
- compromised signing or audit infrastructure;
- side-channel attacks;
- complete formal verification of SQL equivalence;
- a production-grade formal SQL safety proof across all PostgreSQL features;
- cross-database federation;
- full differential privacy for statistical releases;
- arbitrary semantic inference across all allowed query answers.

SessionBound reduces and accounts for what an agent can discover. It does not claim to eliminate all inference and does not provide differential privacy.

IV. SessionBound Model

SessionBound has two major layers: a task control plane and a database runtime. The control plane turns enterprise task approval into a signed task token; the database runtime binds that token to a short-lived credential and enforces safe views, denied fields, budgets, aggregate policy, drift checks, and receipts inside the database session.

Task-bound session state machine. SessionBound treats an approved enterprise task as a stateful database authorization object. A session state is $S = \langle A, C, T, V, L, B, R \rangle$, where A is the approved task, C is the brokered credential, T is the signed token, V is the safe-view registry snapshot, L is the permitted SQL surface, B is the remaining budget vector, and R is the receipt chain. Transitions include issue (approval to token), bind (token and credential to active database state), attempt (SQL shape and safe-view resolution), release (budgeted allowed result), deny (receipt-bearing rejection), and expire/revoke/drift (fail-closed invalidation). The contribution is this state machine around a database session, not a new identity credential, view mechanism, or audit log in isolation.

A. Template, application, approval, and token

A task template is defined by a data platform or application team, not by the agent. It names the task type, purpose, allowed safe views, denied columns, required scope fields, allowed operations, default TTL, budget vector, and aggregate policy. A task application instantiates that template with a requester, actor, concrete scope, and business reason. A task approval decides whether that scoped task should exist; it may be automatic, role-based, or manual, but it does not require the approver to write SQL policies.

The approved task becomes a signed token consumed by the database runtime. In the prototype, the token binds the task id and type, delegator, actor, credential id, tenant and row scope, allowed views, denied columns, permitted operations, query and disclosure budgets, minimum-group aggregate policy, safe-view registry version, policy version, view-definition and exposed-column hashes, audience, expiry, and nonce. The token is structured authorization data, not a natural language prompt.

B. Budgeted database session

A budgeted database session is a database session bound to one active task token. The session may issue open-ended SQL, but every attempt is checked against the token, safe view registry, operation policy, and remaining budget.

In the current prototype, a direct database connection may issue native `SELECT` over approved safe views after binding a signed task token. The PostgreSQL extension checks SQL shape and safe-view OIDs, then executor accounting counts rows and disclosed `expense_id` values before tuples are released. The same path covers SDK `query(sql)`, bound direct `SELECT`, prepared statements, cursor/`FETCH`, `COPY (SELECT)`, and non-`ANALYZE` `EXPLAIN`. Raw table `SELECT` remains ungranted; schema resolution is allowed only so the hook can fail closed and emit receipts. Pooled connections can rebind only with a fresh valid token and can call `unbind_task()` at check-in.

V. Control Plane: Templates, Applications, Approvals, and Budgets

The control plane is the enterprise layer that converts business approval into database-enforceable structure. A data owner approves a task such as "Alice may run a June 2026 Sales travel reimbursement review, excluding salary, bank-account, and phone fields, under a 30-minute TTL and bounded disclosure budget." The approver should not need to author `CREATE POLICY`, `ALTER ROLE`, or `GRANT` statements.

Roles grant eligibility to request task templates, not broad database access. For example, a finance reviewer may request finance-review tasks and a department manager may request department-scoped analysis tasks, but the database ultimately sees a concrete signed task token: actor

X may analyze June 2026 Sales reimbursements through approved views under budget Y. Budget assignment remains a control-plane decision based on role, data classification, task purpose, and approval route; the database runtime enforces the resulting budget.

VI. SessionBoundDB Runtime

SessionBoundDB is the PostgreSQL-based database runtime for SessionBound.

A. Session binding and credential-token binding

The runtime begins with two objects:

1. a short-lived database credential issued by the credential broker;
2. a signed task token issued by the control plane.

The credential authenticates the runtime connection. The task token authorizes the work.

At `bind_task`, the runtime checks:

- the task token signature is valid;
- the task token is unexpired and not revoked;
- the credential is unexpired and not revoked;
- the token `credential_id` matches the current session credential;
- the token `actor` matches the credential actor;
- the session login role matches the broker-issued database user;
- the token audience is `sessionbounddb`;
- no other active task is already bound to this database backend/session.

A production implementation should make a stolen credential without a matching token insufficient, and a stolen token without the matching credential insufficient. Credential-token binding is intended to reduce cross-task credential misuse.

PostgreSQL nuance: inside `SECURITY DEFINER` functions, `current_user` may refer to the function owner. Therefore, a production implementation should use `session_user` and/or an explicit credential registry mapping rather than relying on `current_user` alone. The measured prototype validates token signatures, token expiration, revoked task state, credential-id-to-token matching, actor matching, audience validation, cross-credential token replay rejection, and same-session rebind rejection in the tested prototype path; Section XIII reports those tests explicitly. Production deployments still require hardened credential brokerage, key management, and audit-retention integration outside this demo.

B. Safe views as the query surface

Agents do not query raw application tables. They query registered safe views. Safe views expose business objects, not internal schemas. This is complementary to database-native mechanisms such as PostgreSQL row-level security policies, which provide row filtering at the database layer [6].

Safe views are trusted policy artifacts reviewed by data platform, DBA, and security teams, not agents or ordinary business users. Agents should not have catalog privileges exposing sensitive view definitions. Migrations require review, hash/version updates, and task reapproval; a leaky safe view is a governance failure outside the runtime SQL guard.

Example safe views:

```
expenses
employees
departments
approval_events
ledger_entries
```

A safe view may:

- join raw tables into business objects;
- remove sensitive fields;
- apply tenant, department, month, or project scope;
- expose workflow hints;
- include stable business identifiers for budget accounting.

Safe views are not a complete inference-control solution. They reduce the discoverable surface and make it governable.

C. Operation policy

Native PostgreSQL `SELECT` is the database accounting entrypoint for read-only analytical SQL. The agent-facing SDK exposes this as `query(sql)`: the agent supplies ordinary SQL over approved safe-view names, and PostgreSQL hook and executor paths enforce the task boundary, debit query and disclosure budgets, and emit receipts. `taskbound.run(sql)` remains as a compatibility wrapper. High-value writes should use controlled commands or stored procedures. The runtime rejects mutations, DDL, unsafe utility commands, raw schema access, denied fields, and blocked schemas.

D. Query decision pipeline

At runtime, every SQL attempt follows the deterministic decision procedure in Algorithm 1.

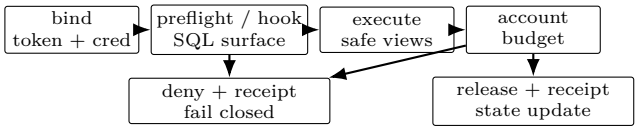

Fig. 2. Bind/query decision pipeline. The wrapper path materializes and checks the candidate result before returning it; the native path intercepts each outgoing tuple and checks/debits it before forwarding.

Algorithm 1. SQL decision pipeline for SessionBoundDB
Input: SQL attempt and bound database session state.
Output: Allowed result with receipt, or denial receipt.

1) Require an active task binding for the session.
2) Validate the signed task token and its TTL.
3) Resolve all referenced relations against registered safe views.
4) Reject denied fields and blocked schemas.
5) Enforce row scope predicates for the bound task.
6) Reject disallowed operation types, including mutations, DDL, and unsafe utility commands.
7) Reserve query budget before execution.
8) Execute only after the structural checks pass.
9) Account result rows and business entities before wrapper release or before each native tuple is forwarded.
10) Emit an allow receipt for a covered release; otherwise reject the attempt and emit a denial receipt.

The runtime does not ask an LLM whether a query is safe.

VII. Security Invariants

The SessionBoundDB enforcement contract can be described by a state

$$S = \langle T, C, V, B, R \rangle, \tag{1}$$

where T is the signed task token, C is the credential/session binding, V is the safe-view registry and policy version, B is the remaining budget vector, and R is the receipt chain. Each SQL attempt is evaluated by:

$$\begin{aligned} &\mathit{decide}(q, S) \rightarrow \mathit{allow}(\mathit{result}, S') \\ &\qquad\qquad \text{or } \mathit{deny}(\mathit{reason}, \mathit{receipt}, S'). \end{aligned} \tag{2}$$

These invariants define the enforcement contract implemented by the prototype and used to structure the adversarial evaluation. They are not a formal proof that all semantic inference is impossible.

- I1. No SQL execution occurs without an active task binding.
- I2. The bound token must match the session credential, actor, audience, expiry, token digest, and revocation state.
- I3. Referenced relations must belong to the approved safe-view registry.
- I4. Direct access to denied fields, raw schemas, catalog escape, mutation, DDL, and blocked payload aggregation is denied.
- I5. Scope predicates or session-bound claims constrain visible rows.
- I6. Budget state is monotonic: an allowed query consumes budget, or the query is denied.
- I7. Every evaluated allowed execution and every bound-runtime denial emits a receipt; connection-level failures before task binding may be logged outside the task receipt chain.
- I8. Safe-view registry, policy version, or definition-hash drift invalidates the token or requires reapproval.
- I9. Direct small-group aggregate release is denied under the configured minimum-group policy.

In the prototype, I1–I2 and I8 are checked during `taskbound.bind_task`; I3–I4 are checked by API-layer AST preflight, the experimental `sessionbound_guard` PostgreSQL hook path, and database permissions; I5 is enforced by safe-view predicates; I6 is implemented through task execution state; I7 is implemented through execution and denial receipts after binding; and I9 is implemented by API shape checks plus wrapper/runtime entity-cardinality checks, with conservative native shape checks for direct SQL. Production implementations should harden these structural checks into always-on planner or executor integration, but the invariant set is independent of that implementation choice.

A. Security Guarantees for the Prototype SQL Fragment

We state the prototype security contract over a deliberately restricted SQL fragment. Let SELECT-F denote single-statement, read-only SQL consisting of `SELECT`, projection, predicates, joins, `GROUP BY/HAVING`, ordering and limits, non-recursive CTEs, and window functions over registered safe-view names. The fragment excludes stacked statements, DDL, DML, `COPY`, `DO`, `CALL`, `CREATE FUNCTION`, temporary object creation, recursive CTEs, set-operation stacking, table functions, raw-schema or catalog references, and high-risk payload aggregation such as `json_agg`, `jsonb_agg`, `array_agg`, `string_agg`, `xmlagg`, `row_to_json`, `json_build_object`, and `jsonb_build_object`. The evaluated `COPY (SELECT)` support is an engineering extension routed through the same relation-resolution and executor-accounting path; the propositions below are stated only for SELECT-F.

Let $\mathrm{Rels}(q)$ be the set of relation identifiers in q after database name resolution and before safe-view expansion. Let $\mathrm{Views}(T, V)$ be the approved safe views carried by token T and validated against registry V. Let $\mathrm{Cols}_{out}(q)$ be the direct column references that appear in the output expressions of q. A state $S = \langle T, C, V, B, R \rangle$ is well formed when the token is valid and bound to credential C, the safe-view registry and policy hashes match the token, the budget vector B is nonnegative, and the receipt chain R verifies. Under these assumptions, the prototype provides the following direct-reference and accounting guarantees. These

are not guarantees against arbitrary semantic inference from allowed answers, and they are not differential privacy.

Proposition 1 (Safe-surface confinement). For any well-formed state S and query $q \in$ SELECT-F, if $\mathit{decide}(q, S) = \mathit{allow}(\mathit{result}, S')$, then $\mathrm{Rels}(q) \subseteq \mathrm{Views}(T, V)$. Consequently, every raw base relation evaluated during query execution is reached only through the definition of a registered safe view approved for the task; the query has no direct reference path to raw tables or catalogs.

Argument. The decision pipeline resolves relation identifiers and checks them against the task's approved safe-view registry before any result is released. Database permissions deny raw-schema access, while the AST preflight and hook path reject resolved relation OIDs outside the approved safe-view set. Therefore any query that names an unregistered relation is denied. Raw base tables may still be read by trusted safe-view definitions, but not by direct agent-supplied SQL.

Proposition 2 (Denied-field non-disclosure for direct references). Let $D(T)$ be the denied-field set in token T, and let $\mathrm{Cols}(v, V)$ be the registered column list for safe view v. If $\mathit{decide}(q, S) = \mathit{allow}(\mathit{result}, S')$, then every direct output column reference $c \in \mathrm{Cols}_{out}(q)$ is in $\bigcup_{v \in \mathrm{Views}(T,V)} \mathrm{Cols}(v, V)$ and $c \notin D(T)$. Thus a column that is absent from the approved safe-view column lists, or that appears in the denied-field set, cannot be directly projected by an allowed query.

Argument. In SELECT-F, output columns must resolve against the relations that survived Proposition 1. A column not exported by an approved safe view cannot be bound by name. A column name or alias in the denied-field set is rejected by the structural policy before execution. The guarantee is syntactic and direct: it does not rule out inferences about denied fields from permitted columns or aggregates.

Proposition 3 (Monotonic budget accounting). Let budgets be ordered componentwise by $\preceq$, and let Δ be the nonnegative budget charge instantiated by the prototype semantics in Table II. For any allowed query, $\mathit{decide}(q, S) = \mathit{allow}(\mathit{result}, S')$ implies $0 \preceq B' \preceq B$, where B' is the budget component of S'. If a wrapper query's materialized candidate result is not covered by the remaining disclosure budget, the wrapper path denies the query before returning result rows. In the native streaming path, each tuple is checked against the in-memory entity budget before that tuple is forwarded; if adding a tuple would exceed the budget, the runtime persists the already accepted prefix and emits a denial receipt before raising an error and before forwarding the over-budget tuple. Thus no tuple is intentionally released without the corresponding nonnegative debit, but native streaming is not an all-or-nothing guarantee for earlier tuples in a query that later exceeds budget.

Argument. The runtime treats budget dimensions as nonnegative counters. Query-count budget is reserved before execution. The wrapper reference path executes into database-owned materialized state, checks minimum-group and disclosure-budget policy over the accounted candidate result, and only then returns rows. The native hook/executor path wraps the destination receiver and observes each outgoing tuple before passing it to the client-side receiver. Both paths subtract only nonnegative charges and record the resulting state in an allow receipt. Therefore the remaining budget cannot increase across an allowed transition; an over-budget transition reaches a denial state at the applicable materialization or tuple-forwarding boundary. In the native path, the destination receiver finalizes charges on normal completion and records a partial-denial prefix if execution aborts after some tuples have passed the budget check.

Proposition 4 (Receipt-chain tamper evidence under trusted DB assumptions). Assume collision-resistant hashing and append-only receipt storage inside the trusted database boundary. For a receipt chain $R = (r_1, \ldots, r_n)$ where each r_i stores the hash of r_{i-1} and a hash of its own canonical contents, removing or modifying any interior receipt r_j, $1 < j < n$, changes r_j's hash or breaks the previous-hash value stored in r_{j+1}. The change is detected by chain verification unless the adversary can violate append-only storage or find a hash collision.

Argument. Each receipt commits to the previous receipt hash and to the current decision, query hash, timestamp, and budget delta. Changing an interior receipt changes its hash; deleting it changes the predecessor expected by the next receipt. Under append-only storage, the adversary cannot rewrite the downstream chain to hide the mismatch. This is a tamper-evidence property for the audit trail under trusted DB assumptions, not a Byzantine storage guarantee against malicious DBAs, storage administrators, compromised hosts, or compromised signing/audit infrastructure.

VIII. Anti-Evasion Scope

SessionBound handles direct boundary violations and accounts for bounded disclosure; arbitrary semantic inference remains outside its formal guarantees.

A. Direct boundary violations

SessionBoundDB can directly block:

- access to raw schemas;
- access to sensitive fields that are not exposed through safe views;
- queries outside row scope;
- unsupported operation types;
- repeated querying beyond task budgets;
- use of credentials without a matching task token;
- attempts to continue after token expiry or revocation.

B. Inference and side channels

SessionBound does not claim to solve arbitrary inference from allowed answers. For example, if a safe view exposes enough correlated information, an agent may infer sensitive facts indirectly. The prototype now denies direct small-group aggregate release under a configurable

minimum-group policy, but arbitrary semantic inference across multiple allowed answers remains outside its formal guarantees. For example, `SUM(amount)` over a large group minus `SUM(amount)` over that group excluding one entity can reveal that entity's contribution even when both groups satisfy `min_group_size`. Future mitigations include predicate-family budgets, aggregate templates, differencing detection, entity-column budgets, repeated-probe penalties, and optional DP for statistical release; these are not implemented here. Timing and resource side channels are also outside the prototype's formal guarantees. Classical database inference-control work studies these risks for statistical databases and repeated query answers [10].

C. Adversarial SQL patterns

Table I summarizes representative evasion patterns, defenses, and limitations.

This table defines the prototype's anti-evasion scope and the path toward production hardening.

IX. Budget Semantics

SessionBound disclosure budgets limit how much task-authorized information an agent may retrieve during one approved analytical session. They are authority accounting, not differential-privacy accounting [11].

A production deployment should use a vector rather than a single unique-row limit. Useful dimensions include query count, result rows, result bytes, per-query payload bytes, aggregate payload shape, unique business entities, column weights, detail-versus-aggregate multipliers, and small-group penalties.

The prototype's unique-row accounting demonstrates the mechanism, while the broader model treats budget as a vector of exposure counters tied to task authority. The current runtime also carries a configurable `min_group_size` policy in the signed task token. For grouped aggregates, direct grouping by sensitive entity identifiers is denied, `HAVING` probes for small groups are denied by default, and the wrapper path checks group cardinality before releasing rows. Aggregate queries over a scoped set are allowed only when the configured entity cardinality meets the policy threshold. Denied small-group queries emit denial receipts and do not release disclosure rows.

Table II gives the exact prototype charge semantics used for Δ. It is intentionally operational: it counts released tuples and stable business identifiers, not information theoretic leakage.

Budget vectors are not mutual-information bounds. They provide operational authority accounting, auditability, and a governance lever for approved enterprise tasks. High-sensitivity deployments can charge entity-column pairs or add repeated-probe penalties.

X. Schema Drift and Safe View Versioning

Task tokens should not silently survive changes to the data boundary they approve. If a safe view changes after approval, the token may no longer represent the approved task.

A task token should therefore bind safe views by more than name: registry version, policy version, database object identifier, view definition hash, and exposed-column hash. PostgreSQL views make this important: `CREATE OR REPLACE VIEW` may preserve `pg_class.oid` while changing the definition, and `relfilenode` is not an appropriate view identity. The prototype binds registry, policy, view-definition, and exposed-column hashes into the signed token and fails closed if the runtime snapshot no longer matches the approved boundary.

XI. Receipts

SessionBound receipts are structured audit records that bind task identity, query decision, and budget impact. They are not merely logs.

Each receipt records the task id, actor, query hash or command name, decision, touched safe views, rows returned, budget delta, remaining budget, timestamp, previous receipt hash, and current receipt hash.

The prototype implements a lightweight hash chain: each receipt stores `previous_receipt_hash` and `receipt_hash`, where the current hash commits to the task id, query hash, decision, budget delta, timestamp, and previous hash. This does not provide a full external proof system, but it detects post-hoc sequence changes within the trusted database boundary under append-only audit storage. It does not protect against malicious DBAs, compromised hosts, storage administrators, or compromised signing/audit infrastructure; DBA-resistant deployments should anchor chain heads to external WORM storage, SIEM, transparency logs, or enterprise audit infrastructure.

XII. Prototype Implementation and Production Hardening

A. Prototype

The current prototype includes:

- FastAPI control plane;
- task template and grant registry;
- short-lived PostgreSQL credential broker;
- signed task token issuance;
- PostgreSQL runtime functions;
- safe views over travel reimbursement data;
- `taskbound.bind_task(payload, signature)`;
- an agent SDK with `TaskboundSession.query(sql)`;
- native guarded safe-view `SELECT` plus compatibility `taskbound.run(sql)`;
- sensitive field denial;
- blocked raw table access;
- query and disclosure budget state;
- receipts for evaluated allowed executions and bound-runtime denial decisions;
- controlled commands for high-value workflow actions;
- an agent-agnostic HTTP evaluation harness.

TABLE I
ADVERSARIAL SQL PATTERNS AND SESSIONBOUND DEFENSES.

Pattern	Example	Defense	Limitation
Raw table escape	`SELECT * FROM app_data.expenses`	no raw grants; safe views only	strong in prototype
Sensitive column	`SELECT salary FROM employees`	denied fields; safe view exclusion	direct leakage only
Scope escape	query another month or department	safe-view predicates from task claims	strong if views are correct
Mutation	`DELETE`, `UPDATE`, `INSERT`	operation policy; read-only `run` path	writes require controlled commands
DDL	`DROP`, `ALTER`, `CREATE`	blocked operations	production should use hook-backed validation
Catalog escape	`pg_catalog`, `information_schema`	blocked schemas and no grants	production needs planner/executor hooks
Pagination scraping	repeated `LIMIT/OFFSET`	query budget and disclosure budget	budget model matters
JSON / array payload aggregation	payload aggregation functions	conservative AST and hook denial; template allowlist is future work	high-risk payload functions are blocked rather than sensitivity-scored
Aggregate inference	small-group `GROUP BY`	configurable minimum group size and entity-aware aggregate policy	arbitrary semantic inference remains out of scope
Timing side channel	expensive predicates	statement timeout and resource budget	partial

TABLE II
PROTOTYPE BUDGET-CHARGE SEMANTICS.

Budget dimension	Prototype charge
`query_count`	Wrapper path: one debit on allowed completion. Native path: one debit reserved before executor run. Preflight and policy denials emit denial receipts without result/entity debit in the current prototype.
`result_rows`	Number of tuples released to the client and recorded in state/receipts. Join multiplicity is visible here, so duplicate output tuples increase this counter.
`unique_entities`	First disclosure of stable business identifiers: new distinct `expense_id` values in released detail tuples for the same `budget_account`. Seen IDs add no entity charge, but still consume `query_count`, `result_rows`, and any bytes/column-weight budgets.
`aggregate_group`	Aggregate rows are allowed only when the configured entity cardinality is at least `min_group_size`; the prototype does not charge every hidden group member as a released detail entity.
payload aggregation	High-risk payload aggregation functions such as `json_agg`, `array_agg`, and `string_agg` are denied unless a future template explicitly allows them.
over-budget native streaming	The over-budget tuple is not forwarded; the accepted prefix is charged and recorded in a denial receipt with prefix rows, new entities, and remaining budget.

B. Implementation honesty

The current hardening prototype uses API-layer AST preflight, PostgreSQL hooks, executor result accounting, rollback-surviving audit writes, and PostgreSQL permissions. Agents do not receive raw table `SELECT` grants. The SDK's `query(sql)` executes native SQL over the bound safe-view schema; a direct `SELECT ... FROM taskbound.expenses` is allowed only after binding and is observed by executor accounting before each tuple is forwarded. The hook is not the sole DDL/DML protection: runtime credentials are deny-by-default and lack raw-table and DDL privileges, while `ProcessUtility_hook` fail-closes and logs utility statements that reach the runtime. The wrapper reference path instead materializes the candidate result and checks group and disclosure policy before returning rows. High-value writes go through controlled commands.

The database hook path is implemented by the `sessionbound_guard` C extension, loaded through `shared_preload_libraries`. The extension registers `post_parse_analyze_hook`, `ProcessUtility_hook`, and executor hooks. Trusted SUSET GUCs populated by `taskbound.bind_task` carry the task, budgets, receipt flags, and approved safe-view OIDs. The executor path wraps the destination receiver to count rows and disclosed `expense_id` values before forwarding tuples. Receipt and budget updates use an autonomous same-database audit channel, so receipts for evaluated allowed executions and hook/API denials survive agent transaction rollback.

The hook rejects non-`SELECT` statements, raw `app_data` relations, catalog access, recursive CTEs, `UNION/INTERSECT/EXCEPT`, unapproved relation OIDs, non-view relations, unsafe non-catalog functions, sensitive output aliases, and high-risk payload aggregation functions such as `json_agg`, `jsonb_agg`, `array_agg`, `string_agg`, `xmlagg`, `row_to_json`, `json_build_object`, and `jsonb_build_object`. The API AST validator remains defense in depth and provides portable preflight feedback before the database runtime is invoked. Minimum-group policy is enforced as API shape checks and wrapper/runtime group-cardinality checks before row release. The native path enforces direct safe-view SQL and tuple-level disclosure accounting, but implements minimum-group protection conservatively: shapes requiring full per-group cardinality reasoning are denied unless routed through the wrapper reference path.

This should still be read as an experimental hook path, not a production SQL-safety proof. A production implementation should harden the same structural policy into:

- always-on PostgreSQL parser/analyzer hooks;
- planner hooks;
- executor hooks;
- extension-level session state and registry management;

TABLE III
IMPLEMENTATION AND EVALUATION PATHS. THE PATHS ARE DELIBERATELY SEPARATED SO THAT WRAPPER REFERENCE OVERHEAD, NATIVE HARDENING COVERAGE, AND HOOK-ONLY STRUCTURAL COST ARE NOT CONFLATED.

Path	Purpose	Enforcement and accounting	Used for	Limitation
Wrapper/API reference path	Compatibility and accounting-complete reference implementation	API AST preflight, database permissions, and `taskbound.run`; PL/pgSQL materialization accounts returned rows and entity exposure; allow/deny receipts are emitted	Canonical validation, adversarial SQL API suite, default p50 overhead, and wrapper scale sweep	Not optimized; 100k-row bottleneck is materialization/accounting
Native PostgreSQL hook/executor path	Production hardening direction for direct safe-view SQL	`post_parse_analyze_hook`, `ProcessUtility_hook`, executor hooks, safe-view OID checks, conservative minimum-group shape checks, row/entity accounting, and receipts	Direct DB enforcement suite, prepared statements, cursor/`FETCH`, `COPY (SELECT)`, `EXPLAIN`, rollback audit, SDK smoke tests, and diagnostic native end-to-end benchmark	Current native accounting is measured but unoptimized; broad production workloads remain future work
Hook-only structural microbenchmark	Isolate parse/analyze structural guard cost	Structural SQL guard only; no row execution, no disclosure accounting, and no receipts	0.125–0.138 ms p50 structural-check cost across SELECT/JOIN/GROUP BY/CTE-window shapes	Not an end-to-end SessionBound performance claim

- deny-by-default schema grants;
- blocked unsafe utility commands;
- executor-level result accounting for deployments that allow native agent `SELECT` syntax, including query-budget debit before execution, bounded result materialization or streaming interception, disclosure-budget accounting before client release, and receipt emission through an audit path that survives transaction aborts;
- `statement_timeout` and resource limits;
- cleanup for expired short-lived roles;
- denial logging outside user transactions.

The measured AST prototype extracts referenced relations, columns, function calls, CTEs, recursive CTEs, subqueries, joins, catalog access, operation type, and set operations. It blocks raw schema references, catalog access, mutations, DDL, `COPY`, `SET`, `SHOW`, `CREATE FUNCTION`, `DO` blocks, recursive CTEs, `UNION`/`INTERSECT`/`EXCEPT` stacking, unknown functions, denied-field aliases, and high-risk payload aggregation functions. The API AST validation evaluation passed 20 of 20 query-shape cases, and the PostgreSQL hook evaluation passed 18 of 18 direct database enforcement cases.

C. Query example

Allowed:

```
WITH ranked AS (
  SELECT expense_id, department_name, category, amount,
         row_number() OVER (
           PARTITION BY department_name ORDER BY amount DESC
         ) AS rn
  FROM expenses
  WHERE expense_month = '2026-06'
)
SELECT department_name, expense_id, category, amount
FROM ranked
WHERE rn = 1;
```

Denied:

```
SELECT employee_name, salary FROM employees;
```

Denied:

```
SELECT * FROM app_data.expenses;
```

Denied:

```
DELETE FROM expenses WHERE expense_month = '2026-06';
```

XIII. EVALUATION

The prototype evaluation has two goals:

1. validate security correctness and boundary behavior for approved and adversarial SQL;
2. diagnose performance and production feasibility across the wrapper reference path, stronger baselines, and native structural checks.

The evaluation focuses on functional boundary validation, security-property comparisons against narrower database controls, database-runtime overhead, and concurrency behavior on the default synthetic dataset.

A. Experimental setup

The reported evaluation used Docker Compose with PostgreSQL 16.14, the FastAPI SessionBoundDB prototype, and the `sessionbound_guard` extension loaded through `shared_preload_libraries`. The default seed contains 430 expenses, 240 employees, and 124 departments. The artifact metadata records the git commit identifier, environment variables, raw result paths, and run timestamps.

Experiments ran on Ubuntu 22.04.5 LTS under WSL2 on a 13th Gen Intel Core i9-13900HX host with 32 logical CPUs and 15 GiB RAM, using Docker 29.1.3, Docker Compose v2.40.3, and Docker Desktop `overlay2` storage. PostgreSQL used default settings except for `shared_preload_libraries`. API measurements used local Docker networking; database-only measurements used direct connections from the Compose network. The benchmark harness did not intentionally generate competing load.

The AST and adversarial SQL evaluations exercise the HTTP API, including dynamic credential issuance, signed task-token creation, token binding, AST preflight, the wrapper reference query path, budget state, and receipts. A separate SDK surface evaluation calls `TaskboundSession.query(sql)` directly from the Compose network. The hook evaluation bypasses the API query

endpoint for agent cases by using generated runtime credentials directly against PostgreSQL, and it uses trusted superuser setup for hook-only structural checks. The overhead breakdown uses direct PostgreSQL connections from the Compose network so that it isolates database runtime overhead from HTTP, model calls, dynamic credential creation, and API-layer AST parsing. A hook-only microbenchmark measures `sessionbound_guard_check(sql)` directly, exercising PostgreSQL parse/analyze and the structural guard without executing result rows or performing wrapper materialization.

B. Artifact reproducibility

The artifact is container-oriented and includes code, synthetic data, raw outputs, and LaTeX source. A reviewer starts the database, extension, API, and seed data with `docker compose up -d --build`, then runs the scripts in `paper/tdsc/scripts/` against `localhost:8000` and PostgreSQL on `localhost:15432`. The reproduction entrypoints are `ast_validation_eval.py`, `adversarial_sql_eval.py`, `sessionbound_guard_hook_eval.py`, `overhead_breakdown.py`, `hook_microbenchmark.py`, `rollback_audit_eval.py`, and `native_end_to_end_benchmark.py`. A targeted `native_partial_budget_eval.py` validates native over-budget prefix accounting; `concurrent_isolation_eval.py` validates task isolation. Exact command lines and environment variables are recorded in the artifact README and changelog.

The artifact manifest maps each table to its raw JSON/CSV result files. Scripts write timestamps, git commit, iteration counts, error counts, and measured latencies into those files; the tables below are derived from raw outputs rather than hand-entered experimental claims.

C. Functional validation

We evaluated the SessionBoundDB PostgreSQL prototype using Docker Compose. The artifact contains the evaluation scripts, validation reports, benchmark outputs, and source code needed to reproduce the results. The canonical validation suite passed 24 of 24 scenarios; the full scenario matrix and raw receipts are provided in the artifact repository rather than reproduced in the main text.

Scope violations over safe views are enforced by filtering rather than explicit denial. A query for another approved-view but out-of-scope month returns zero rows because the safe view predicate binds the session to the task scope. The measured prototype conservatively blocks payload aggregation functions such as `json_agg`, `array_agg`, `string_agg`, and `row_to_json`; ordinary aggregate analysis such as `count`, `sum`, `avg`, `min`, and `max` remains allowed when it satisfies the configured minimum-group policy.

D. Baseline security comparison

We compare SessionBound against four narrower database configurations to separate access-surface reduction from task-bound session enforcement:

- **Raw PostgreSQL**: direct access to application tables using a trusted analyst/admin test role.
- **Role-only**: a constrained database role with coarse grants but no task token, budget vector, receipt chain, or task-specific safe view binding.
- **Safe-view-only**: curated views and denied-field exclusion, without credential-token binding, query budgets, disclosure budgets, or signed task tokens.
- **RLS + Safe View + Short Credential + Audit**: PostgreSQL RLS policies over raw tables, field-limited safe views, a short-lived read-only role, and basic query-hash/actor/timestamp/row-count audit logging, but no task token, credential-token binding, disclosure budget, receipt hash chain, or safe-view token invalidation.
- **SessionBound full**: signed task token, short-lived credential binding, safe views, denied fields, operation policy, query budget, disclosure budget, minimum-group policy, anti-payload aggregation checks, and receipts.

For fairness, the safe-view-only baseline uses pre-scoped or manually parameterized views matching the evaluated task scope, but it lacks signed task binding, cumulative budgets, drift invalidation, and decision-bound receipts. The RLS+Safe View+Short Credential+Audit baseline is intentionally strong and may use pre-scoped or protected trusted-session predicates. Even then, it lacks signed task-token binding, credential-token matching, cumulative disclosure budget, safe-view drift invalidation, and decision-bound receipts. A client-writable RLS scope parameter is unsafe; a protected one becomes bespoke control-plane/runtime plumbing rather than plain RLS alone.

Table VI summarizes the observed security properties. In the updated credential-token test suite, SessionBound denies credential-id mismatch, cross-credential token replay, wrong audience, wrong actor, expired tokens, revoked tasks, and same-session rebind to a second active task. It also rejects tokens whose safe-view registry version, policy version, view-definition hash, or exposed-column hash no longer matches the runtime registry.

E. Adversarial SQL evaluation

The reported adversarial evaluation uses an SQL suite with 140 cases. The suite covers direct boundary violations, raw-schema escape, catalog and metadata escape, `pg_temp`/search-path/GUC tampering, function abuse, payload aggregation and compression, JSON/XML/composite leakage, prepared-statement and cursor lifecycles, `COPY` and `EXPLAIN` variants, CTEs, recursive CTEs, set operations, `VALUES`/`LATERAL`/`DISTINCT ON`/table sampling, DDL/DML/utility commands, aggregate-inference probes, pagination and budget scraping, revocation/expiry/rebind attempts, schema drift, and rollback/audit-survival cases. The suite passed all expected classifications: 126 cases were blocked and 14 safe-view analytical cases were allowed and accounted for. All tested

direct small-group aggregate-release attempts were blocked by the configured minimum-group policy.

This result should not be read as a proof of complete SQL safety. It does show that the current prototype is no longer limited to literal string checks for the main tested attack families. Minimum-group policy mitigates direct small-group aggregate release, while arbitrary semantic inference across multiple allowed answers remains out of scope.

We separately evaluated the native SDK surface. A smoke test confirmed that `TaskboundSession.query(sql)` and bound direct `SELECT ... FROM taskbound.expenses` are both accounted, while the same query after `unbind_task()` fails closed.

We also evaluated the database hook path directly. The `sessionbound_guard` suite passed 18 of 18 cases. It confirmed GUC tamper resistance, allowed bound native `SELECT`, support for prepared statements, cursor/`FETCH`, `COPY (SELECT)`, and non-`ANALYZE` `EXPLAIN`, and blocked set operations, catalog/raw access, runtime-helper abuse, payload aggregation, unapproved OIDs, `HAVING`/direct-entity aggregate shapes, and `EXPLAIN ANALYZE`.

F. Performance overhead and ablation

The latest overhead breakdown measures five query patterns: simple `SELECT`, `JOIN`, `GROUP BY`, CTE, and window function. Each pattern uses 10 warmup iterations and 100 measured iterations per mode. The run compares raw PostgreSQL, role-only, safe-view-only, the RLS+safe-view+short-credential+audit baseline, SessionBound without receipts, SessionBound without budget updates, and full SessionBound. Table VII reports p50 latency in milliseconds; all listed measurements completed with zero query errors.

Raw, role-only, and safe-view-only queries are sub-millisecond on the default seed. The stronger RLS+safe-view+short-credential+audit baseline adds a basic audit insert and measures 0.98–1.12 ms p50. Full SessionBound measures 17.4–18.6 ms p50 across the five patterns. Disabling receipts or budget accounting does not consistently reduce latency, so receipt insertion and budget updates do not dominate this small-dataset benchmark. The remaining cost is primarily the wrapper reference path's session/runtime checks and PL/pgSQL materialization.

1) Performance diagnosis: The database-runtime breakdown isolates direct PostgreSQL execution and therefore does not include HTTP latency, model calls, dynamic credential creation, or API-layer AST preflight. End-to-end agent requests do incur the AST preflight before entering PostgreSQL, so the end-to-end path is strictly larger than the database-only numbers in Table VII.

Within the evaluated wrapper reference path, the overhead comes from several implementation layers in the wrapper reference prototype. First, `taskbound.run` dispatches each query through a PL/pgSQL wrapper rather than through PostgreSQL's native planner and executor path alone. Second, each execution repeatedly resolves session claims and task state instead of caching a bound task descriptor in trusted backend state. Third, safe views add predicate and view-expansion cost even before SessionBound receipt or budget logic runs. Fourth, the wrapper path materializes result rows so that the runtime can account for returned business identifiers. Fifth, receipt insertion and budget accounting add state updates, but the no-receipt and no-budget ablations show that these updates are not the dominant source on the small default dataset.

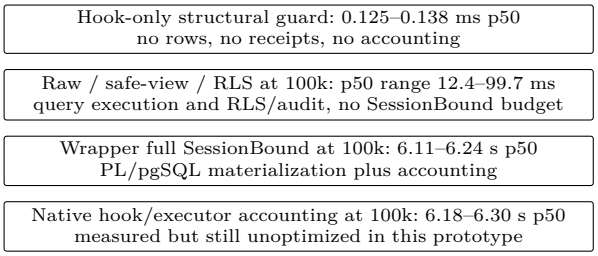

Fig. 3. Performance diagnosis. Structural guarding is sub-millisecond, while both accounting-complete paths become multi-second at 100k rows; the current bottleneck is result accounting/materialization rather than parse/analyze checks.

To separate structural enforcement from wrapper-era execution cost, we also ran a hook-only microbenchmark over the native `sessionbound_guard_check(sql)` path. This path prepares SQL through PostgreSQL parse/analyze and the structural guard, but does not execute result rows, perform executor row accounting, insert receipts, or materialize JSON rows through `taskbound.run`. With 20 warmup and 200 measured iterations per query shape, allowed structural checks had p50 latency of 0.125 ms for SELECT, 0.131 ms for JOIN, 0.131 ms for GROUP BY, and 0.138 ms for CTE/window SQL; raw-schema and UNION denial checks also passed. This is not an end-to-end performance claim for the native executor-accounting path, but it shows that structural parse/analyze guarding is not the source of the 100k-row multi-second behavior.

The diagnostic native end-to-end benchmark in Table VIII measures five query shapes over 1k, 10k, and 100k scoped rows. The SessionBound modes use API-issued runtime credentials and signed task tokens, but API calls are outside measured query latency. The wrapper mode executes through `taskbound.run(sql)`. The native mode binds the same task token and then issues direct safe-view SQL through the PostgreSQL hook and executor-accounting path.

The result is intentionally conservative. Safe-view-only and RLS+safe-view+audit baselines remain within roughly 118 ms p99 at 100k rows across these query shapes. Both current SessionBound accounting paths are much slower: wrapper p50 is 6.11–6.24 s and native hook/executor p50 is 6.18–6.30 s at 100k rows. Thus the hook-only structural result should not be presented as full SessionBound performance. The current native accounting path demonstrates direct DB enforcement semantics but has not yet delivered

TABLE IV
ADVERSARIAL SQL EVALUATION SUMMARY FROM THE 140-CASE HARDENING SUITE.

Bucket	Representative pattern	Result	Notes
Blocked direct violations	denied columns, raw schema, catalogs, GUC/session tampering, unsafe functions, prepared/cursor misuse, unsafe `COPY` variants, raw-table `COPY`, `EXPLAIN ANALYZE`, set operations, DDL/DML, replay/drift/rollback attempts	115 blocked / 115	direct attempts to leave the approved task boundary denied
Blocked payload aggregation	`json_agg`, `array_agg`, `string_agg`, XML/JSON builders, row/composite packing	6 blocked / 6	high-risk one-row payload compression denied by default
Blocked small-group aggregate	direct entity grouping, `HAVING` probes, filtered small group, high configured k	5 blocked / 5	direct small-group aggregate release denied
Allowed safe-view analytics	ordinary safe-view projection, join, aggregate, CTE/window, bounded pagination, harmless aliasing	14 allowed / 14	allowed cases emitted receipts and were budget-accounted
Total	all adversarial cases in `adversarial_sql_20260708_235742.json`	140 passed / 140	zero expected-classification failures

TABLE V
POSTGRESQL HOOK AND EXECUTOR ENFORCEMENT EVALUATION. AGENT CASES CONNECT DIRECTLY AS THE GENERATED RUNTIME CREDENTIAL AND ISSUE NATIVE SQL AFTER BINDING A SIGNED TASK TOKEN.

Case group	Enforcement path	Result	Notes
Trusted context	Non-superuser direct DB session	1 blocked / 1	Ordinary agent role cannot set SUSET guard GUCs
Native allowed surface	Agent credential direct DB session	6 allowed / 6	Bound `SELECT`, schema-qualified safe-view query, prepared `EXECUTE`, cursor/`FETCH`, `COPY (SELECT)`, and non-`ANALYZE` `EXPLAIN` accounted
Native blocked surface	Agent credential direct DB session	5 blocked / 5	`UNION`, catalog access, recursive CTE, private helper access, and `EXPLAIN ANALYZE` denied
Hook-only structural checks	Trusted-GUC hook check without API preflight	6 blocked / 6	Non-`SELECT`, raw schema, payload aggregation, unapproved safe-view OID, direct entity `GROUP BY`, and `HAVING` denied

TABLE VI
SECURITY-PROPERTY COMPARISON ACROSS EVALUATED CONFIGURATIONS. THE RLS BASELINE INCLUDES ROW POLICY, FIELD-LIMITED VIEWS, SHORT CREDENTIALS, READ-ONLY GRANTS, AND AUDIT, BUT OMITS SESSIONBOUND TASK TOKENS, BUDGETS, DRIFT INVALIDATION, AND RECEIPT CHAINS.

Property	Raw	Role	Safe view	RLS+SV+Audit	SessionBound
Blocks writes	No	Yes	Yes	Yes	Yes
Blocks raw table access	No	No	Yes	Yes	Yes
Blocks denied fields	No	No	Yes	Yes	Yes
Enforces row scope	No	No	Yes	Yes	Yes
Query budget	No	No	No	No	Yes
Disclosure budget	No	No	No	No	Yes
Payload aggregation blocking	No	No	No	No	Yes
Credential-token binding	No	No	No	No	Yes
DB-local receipt hash chain	No	No	No	No	Yes
Basic audit log	No	No	No	Yes	Receipts
Schema drift token invalidation	No	No	No	No	Yes

optimized scale behavior.

The 100k-row setting intentionally raises the disclosure budget to stress-test accounting completeness for large detail releases. It should not be interpreted as the common operating mode for approved analytical tasks, where detail-release budgets are typically smaller and aggregate queries dominate. Enterprise audits may scan 100k+ rows without releasing 100k detail rows to an agent; production deployments should optimize aggregate-heavy executor accounting and route high-cardinality detail exports through controlled workflows. The roadmap is cached task descriptors, executor accounting, `DestReceiver`/CustomScan-style in-

TABLE VII
OVERHEAD P50 LATENCY IN MILLISECONDS. EACH PATTERN/MODE USED 10 WARMUP AND 100 MEASURED ITERATIONS.

Pattern	Raw	Role	Safe view	RLS+SV+Audit	SB no receipt	SB no budget	SB full
SELECT	0.230	0.227	0.233	1.067	17.075	16.699	17.397
JOIN	0.201	0.181	0.209	0.976	18.712	18.197	18.616
GROUP BY	0.229	0.230	0.272	1.100	18.098	17.125	18.289
CTE	0.252	0.238	0.254	1.045	16.914	17.598	17.698
Window	0.251	0.238	0.281	1.119	16.818	17.778	17.639

tegration, aggregate-first accounting, and less PL/pgSQL materialization.

G. Scale and concurrency

We separately tested credential-token edge cases. The updated prototype denied all tested edge cases: credential A with token B, replaying a token with a second credential, binding a second active task in the same session, wrong token audience, wrong token actor, expired token, and revoked task state. We also tested safe-view drift checks: registry version drift, safe-view policy-version drift, view-definition hash mismatch, and exposed-column hash mismatch were denied and require re-approval before execution.

The default seed contains 430 expenses, 240 employees, and 124 departments. A concurrency smoke test over the API completed 1, 5, and 20 concurrent sessions with zero failed sessions. At 20 concurrent sessions, query p50 was 82.896 ms and query p95 was 110.356 ms. This is an API-stack smoke test, not a throughput-capacity claim. The concurrent isolation script confirmed independent credentials, budgets, and task-specific receipts for two active tasks, and denied credential A with token B and same-backend double binding.

We also ran a synthetic scale sweep over 1k, 10k, and 100k scoped expense rows. Table VIII reports p50 ranges across `SELECT`, `JOIN`, `GROUP BY`, CTE, and window queries, along with the maximum p95 and p99 observed across those patterns. All listed measurements completed with zero query errors. The 100k target is a detail-release stress setting with elevated budgets; these results expose a scale-sensitive limitation shared by the current wrapper and native accounting implementations and should not be read as a production-throughput claim.

TABLE VIII
DIAGNOSTIC END-TO-END SCALE BENCHMARK IN MILLISECONDS. P50 IS THE RANGE ACROSS FIVE QUERY SHAPES; P95/P99 ARE MAXIMA. THE 1K AND 10K TARGETS USE 10 WARMUP PLUS 100 MEASURED ITERATIONS; 100K USES 10 WARMUP PLUS 30 MEASURED ITERATIONS. THE 100K SETTING USES ELEVATED DISCLOSURE BUDGETS AS A DETAIL-RELEASE STRESS TEST, NOT THE EXPECTED COMMON OPERATING MODE.

Rows	Mode	p50 range	max p95	max p99	Err.
1k	Raw	0.43–1.16	1.28	1.34	0
1k	Safe view	0.63–1.22	1.36	1.49	0
1k	RLS+SV+Audit	1.32–2.06	2.82	3.29	0
1k	SB wrapper	75.63–92.57	104.39	111.13	0
1k	SB native	79.80–95.53	106.71	113.85	0
10k	Raw	2.03–8.76	9.68	10.63	0
10k	Safe view	2.82–9.48	10.27	11.08	0
10k	RLS+SV+Audit	3.70–10.24	11.69	12.38	0
10k	SB wrapper	632.57–646.97	686.76	730.08	0
10k	SB native	632.39–644.28	691.20	726.82	0
100k	Raw	12.41–93.42	102.18	103.88	0
100k	Safe view	24.14–97.80	112.63	117.71	0
100k	RLS+SV+Audit	17.14–99.71	109.32	109.52	0
100k	SB wrapper	6108.33–6236.90	6531.86	6626.32	0
100k	SB native	6176.94–6298.30	6537.96	6690.70	0

H. Discussion of remaining gaps

Several gaps remain. The PostgreSQL hook path now includes native executor accounting, but the disclosure unit is still demo-specific and the native path is not optimized across broad production workloads. Minimum-group policy mitigates direct small-group aggregate release, but arbitrary semantic inference across multiple allowed answers remains out of scope; the budget vector is operational and does not provide formal differential privacy.

XIV. Related Work

SessionBound is a task-session contract, while prior systems are identity-, policy-, product-, operation-, or audit-centered. RLS, short-lived credentials, safe views, and audit logs implement important pieces, but do not by themselves bind row scope, credential identity, SQL surface, cumulative budget, drift checks, and decision-bound receipts into one approved database session.

ABAC, XACML, capabilities, purpose-aware access, OAuth, authenticated delegation, and Zanzibar provide authorization, delegation, or relationship checks [12], [13], [14], [15], [16], [17], [1], [2], [9]; SessionBound addresses how the resulting SQL session is scoped, budgeted, drift-checked, and audited.

PAuth authorizes concrete service or tool operations implied by a natural-language task [3]. SessionBound is complementary: it assumes approval is compiled into structured claims, then enforces a multi-query SQL session with safe views, budgets, drift checks, and receipts.

Data Product MCP governs discovery, access, and contract-aware querying of enterprise data products [4], [5]. SessionBound can consume those outputs, but enforces the lower-level PostgreSQL execution contract after approval. Prompt-injection work, information-flow controls, RLS/VPD/DDS, parser-mediated validation, auditing, inference control, and differential privacy address adjacent pieces [18], [19], [20], [21], [22], [6], [7], [8], [23], [24], [25], [26], [10], [11]. SessionBound composes these ideas around a task-bound database session; its disclosure budgets are operational accounting, not formal DP.

XV. Limitations

SessionBound is a research prototype and reference architecture, not a formal SQL safety proof. It targets one PostgreSQL database; production deployments need KMS/JWKS signing, credential lifecycle management, safe-view migration review, external audit retention, and WORM anchoring. Disclosure budgets are operational authority accounting, not differential privacy or complete semantic inference control: minimum-group policy mitigates direct small-group aggregate release, but arbitrary inference across multiple allowed answers remains out of scope and high-risk payload aggregation is conservatively blocked. The measurements cover microbenchmarks, overhead ablations, concurrency smoke tests, and 1k/10k/100k diagnostic scale benchmarks, not optimized broad production workloads. Federation, malicious DBAs, and compromised hosts are out of scope.

XVI. Conclusion

SessionBound turns enterprise task approval into budgeted database sessions for AI agents: "The agent can try. The database decides." It binds approval, credential identity, safe-view versioning, SQL surface, budgets, and receipts into one database-session contract. The prototype shows that structural parse/analyze guarding is sub-millisecond, while current accounting-complete paths become multi-second at 100k rows; the production path is to optimize result accounting, backend state caching, and PostgreSQL executor integration without weakening that contract.

References

[1] D. Hardt, "The OAuth 2.0 Authorization Framework," RFC 6749, 2012. [Online]. Available: https://www.rfc-editor.org/info/rfc6749

[2] T. South, S. Marro, T. Hardjono, R. Mahari, C. D. Whitney, D. Greenwood, A. Chan, and A. Pentland, "Authenticated Delegation and Authorized AI Agents," 2025. [Online]. Available: https://arxiv.org/abs/2501.09674

[3] R. K. Sharma, L. Jiang, Z. Lin, and S. Chen, "PAuth - Precise Task-Scoped Authorization For Agents," 2026. [Online]. Available: https://arxiv.org/abs/2603.17170

[4] M. Tonnarelli, F. Scaramuzza, S. Harrer, and L. W. Dietz, "Data Product MCP: Chat with your Enterprise Data," 2026. [Online]. Available: https://arxiv.org/abs/2601.08687

[5] Model Context Protocol, *Model Context Protocol Specification*, 2025. [Online]. Available: https://modelcontextprotocol.io/specification/2025-06-18

[6] PostgreSQL Global Development Group, *Row Security Policies*, PostgreSQL, 2026. [Online]. Available: https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[7] Oracle, *Using Oracle Virtual Private Database to Control Data Access*, Oracle, 2026. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/19/dbseg/using-oracle-vpd-to-control-data-access.html

[8] ——, *What Is Oracle Deep Data Security*, Oracle, 2026. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/26/ddscg/what-is-oracle-deep-data-security.html

[9] R. Pang, R. Caceres, M. Burrows, Z. Chen, P. Dave, N. Germer, A. Golynski, K. Graney, N. Kang, L. Kissner, J. L. Korn, A. Parmar, C. D. Richards, and M. Wang, "Zanzibar: Google's Consistent, Global Authorization System," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, 2019, pp. 33–46. [Online]. Available: https://www.usenix.org/conference/atc19/presentation/pang

[10] N. R. Adam and J. C. Wortmann, "Security-Control Methods for Statistical Databases: A Comparative Study," *ACM Computing Surveys*, vol. 21, no. 4, pp. 515–556, 1989. [Online]. Available: https://doi.org/10.1145/76894.76895

[11] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*, ser. Foundations and Trends in Theoretical Computer Science. Now Publishers, 2014, vol. 9. [Online]. Available: https://doi.org/10.1561/0400000042

[12] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to Attribute Based Access Control (ABAC) Definition and Considerations," National Institute of Standards and Technology, Special Publication 800-162, 2019. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-162

[13] OASIS XACML Technical Committee, *eXtensible Access Control Markup Language (XACML) Version 3.0*, OASIS, 2013. [Online]. Available: https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html

[14] J. B. Dennis and E. C. Van Horn, "Programming Semantics for Multiprogrammed Computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966. [Online]. Available: https://doi.org/10.1145/365230.365252

[15] M. S. Miller, K.-P. Yee, and J. Shapiro, "Capability Myths Demolished," Johns Hopkins University Systems Research Laboratory, Tech. Rep. SRL2003-02, 2003. [Online]. Available: https://classpages.cselabs.umn.edu/Fall-2021/csci5271/papers/SRL2003-02.pdf

[16] R. Agrawal, J. Kiernan, R. Srikant, and Y. Xu, "Hippocratic Databases," in *Proceedings of the 28th International Conference on Very Large Data Bases*, 2002, pp. 143–154. [Online]. Available: https://www.vldb.org/conf/2002/S05P02.pdf

[17] J.-W. Byun and N. Li, "Purpose Based Access Control for Privacy Protection in Relational Database Systems," *The VLDB Journal*, vol. 17, no. 4, pp. 603–619, 2008. [Online]. Available: https://doi.org/10.1007/s00778-006-0023-0

[18] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023. [Online]. Available: https://doi.org/10.1145/3605764.3623985

[19] Y. Liu, G. Deng, Y. Li, K. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, "Prompt Injection Attack against LLM-integrated Applications," 2023. [Online]. Available: https://arxiv.org/abs/2306.05499

[20] OWASP GenAI Security Project, *LLM01:2025 Prompt Injection*, OWASP, 2025. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[21] A. C. Myers and B. Liskov, "A Decentralized Model for Information Flow Control," in *Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles*, 1997, pp. 129–142. [Online]. Available: https://www.cs.cornell.edu/andru/papers/iflow-sosp97/paper.html

[22] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI Agents with Information-Flow Control," 2025. [Online]. Available: https://arxiv.org/abs/2505.23643

[23] Oracle, *Oracle Deep Data Security Technical Brief*, Oracle, 2026. [Online]. Available: https://www.oracle.com/a/ocom/docs/security/deep-data-security-technical-brief.pdf

[24] S. W. Boyd and A. D. Keromytis, "SQLrand: Preventing SQL Injection Attacks," in *Applied Cryptography and Network Security*. Springer, 2004, pp. 292–302. [Online]. Available: https://doi.org/10.1007/978-3-540-24852-1_21

[25] Z. Su and G. Wassermann, "The Essence of Command Injection Attacks in Web Applications," in *Proceedings of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 2006, pp. 372–382. [Online]. Available: https://doi.org/10.1145/1111037.1111070

[26] J. Kleinberg, C. Papadimitriou, and P. Raghavan, "Auditing Boolean Attributes," in *Proceedings of the Nineteenth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 2000. [Online]. Available: https://doi.org/10.1145/335168.335210